

APIConnect-catalog

How the GitOps pipeline for IBM API Connect works — a practical guide

TARGET PLATFORM

IBM API Connect 10.0.8.x — Enterprise as a Service (SaaS)

DataPower API Gateway

REPOSITORY

APIConnect-catalog (this bundle)

AUDIENCE

Platform / integration engineers who operate the pipeline, and API developers who submit changes

CI/CD

Azure DevOps Pipelines, self-hosted Linux x64 agent (on-prem) or Microsoft-hosted

DOCUMENT VERSION

1.0 — 27 September 2026

COMPANION FILES

README.md (reference), .azure-pipelines/, scripts/, tools/

This document explains the design and the day-to-day use. The README in the repository is the concise technical reference; where they differ, the repository files are authoritative.

Contents

1. Why this exists — the problem and the three decisions
2. The big picture — from pull request to production
3. API Connect concepts you need — API, Product, Plan, Catalog
4. Repository layout — where things live and why
5. Life of a change — step by step
6. The four validation stages
7. Backward compatibility and versioning policy
8. Publishing and promotion — dev, test, prod
9. One definition, three environments — catalog-specific properties
10. Security — identities, secrets, permissions
11. Setting up an on-prem agent from the offline bundle
12. Setting up Azure DevOps
13. Day-2 operations — add, change, break, retire, roll back
14. Troubleshooting — reading a failed run
15. Before the first publish — items to verify on your tenant
16. Appendix — scripts, tools and versions

Reading guide

Developers who only need to submit API changes can read sections 2, 5, 6, 7 and 13. Operators should read everything once; sections 11, 12 and 15 are the set-up checklist.

1. Why this exists — the problem and the three decisions

Publishing APIs to API Connect by hand — editing drafts in API Manager and pressing *Publish* — works for one team and a handful of APIs. It stops working when several teams share an environment: nobody knows what is live, changes cannot be reviewed before they hit the gateway, breaking changes slip through, and dev, test and prod drift apart.

GitOps turns this around. The Git repository *is* the catalog. Every API and every Product is a YAML file. A change is a pull request that is validated automatically. When it is merged, a pipeline logs in with the `apic` toolkit and publishes exactly what is in Git — first to `dev`, then, after approval, to `test` and `prod`.

The three design decisions

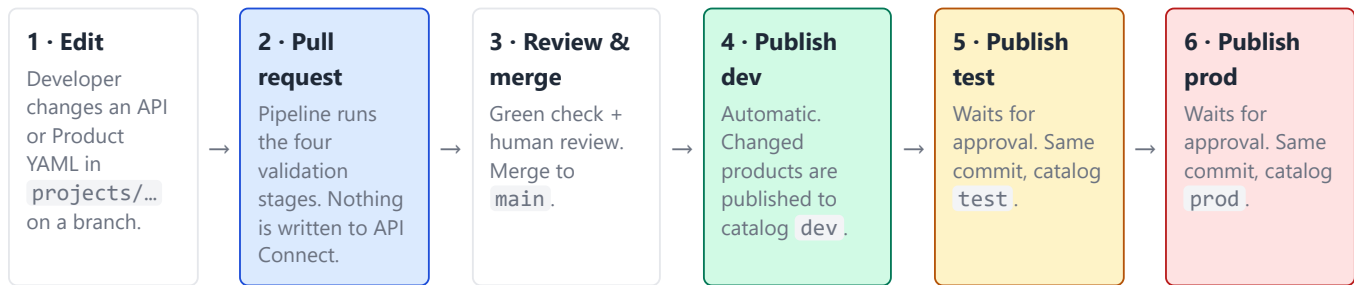
Question	Decision	Why
Should Products (plans, rate limits, visibility) be managed in Git too, or left to API Manager's auto-publish?	Yes, always. Every API sits next to a Product file that references it. The pipeline publishes Products, never bare APIs.	In API Connect v10 an API is only reachable on the gateway through a Product published to a Catalog, and the toolkit has no <code>apis:publish</code> — <code>products:publish</code> is the only publish command. Auto-publish creates opaque products you do not control.
Publish on pull request or on merge ?	PR = validate only; merge = publish. PR builds never write to API Connect.	Two open PRs touching the same product would fight over the same <code>name:version</code> in a shared catalog. Validation is enough to give fast feedback; publishing belongs to the merged, reviewed state.
How to promote dev → test → prod: branches, folders or configuration?	Configuration. One <code>main</code> branch, one file per API, environment differences expressed inside the file.	A branch per environment drifts and needs cherry-picks; a folder per environment duplicates every file. With configuration, test and prod receive byte-identical artifacts and the pipeline stage decides the target catalog.

Principles that follow from this

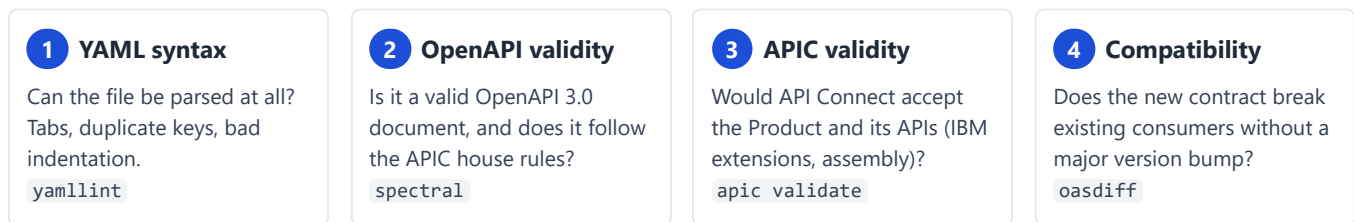
- Git is the single source of truth; API Manager becomes read-only for humans (viewer roles). Only the pipeline identity publishes.
- Drafts in API Manager are not used at all — the PR *is* the draft.
- The pipeline never deletes, deprecates or retires anything. Removing an API from the gateway is a deliberate manual step.
- A breaking change is not forbidden — it just has to arrive as a new major version that coexists with the old one.

2. The big picture — from pull request to production

Everything starts with a developer editing a YAML file. The diagram shows the whole path.



The four validation stages (fail fast, in this order)



If a stage fails, the later ones do not run: the developer sees the first real problem, not a wall of consequences. All findings appear as annotations on the build and as test results in the pull request's *Checks* tab.

What is published, exactly?

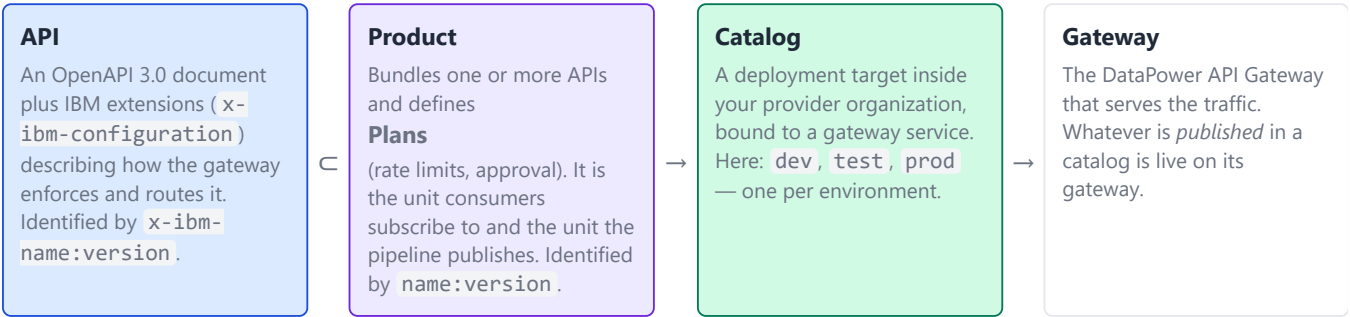
Only what changed. The pipeline computes a **change set**: every Product file that was modified, plus every Product that references a modified API file. Each environment remembers the last commit it received through a Git tag (`published/dev`, `published/test`, `published/prod`); the change set for an environment is “everything since its tag”. That is why a failed prod publish on Monday is simply picked up by Tuesday's run, and why the very first run (no tags yet) publishes the whole repository — the bootstrap.

Where does the environment-specific configuration go?

Inside the API file, under `x-ibm-configuration.catalogs.<catalog>.properties`. The gateway resolves the value for the catalog it runs in, so one file serves dev, test and prod (section 9).

3. API Connect concepts you need

Four objects matter for this pipeline. If you are new to API Connect, this page is enough to follow the rest of the document.



Term	What it means for you
Provider organization	Your tenant's top-level container. There is exactly one; its name is the <code>APIC_ORG</code> variable.
Catalog	Environment. The pipeline stage chooses the catalog; the folder structure never encodes it.
Space	An optional sub-partition of a catalog with its own permissions. Not used today. If enabled later, each <code>projects/<name></code> folder maps to a Space.
Plan	Part of a Product: rate limits and whether subscriptions need approval. Consumers subscribe to a plan.
Publish vs. stage	Publishing makes a product live on the gateway. Staging uploads it without activating it. This pipeline always publishes.
Republish	Publishing a product <code>name:version</code> that already exists updates it in place; subscriptions are kept. Used for non-breaking changes.
Replace / supersede	Toolkit operations that move subscriptions from an old product version to a new one. Used manually after a breaking (major) change.

The one rule people forget

Changing `x-ibm-name` makes API Connect see a *new* API; the old one stays published until someone retires it. Treat `x-ibm-name` as permanent, and use `info.version` to evolve.

4. Repository layout

APIConnect-catalog/	
├─ .azure-pipelines/	
│ └─ apic-ci.yml	the pipeline: validate → publish dev → test → prod
│ └─ templates/	
│ └─ install-tools.yml	tooling; online (hosted agent) or offline (on-prem agent)
│ └─ publish-stage.yml	one deployment stage per catalog
├─ config/	
│ └─ .yamllint.yml	stage 1 rules
│ └─ spectral.yml	stage 2 rules: OpenAPI 3.0 + APIC house rules
│ └─ compat-ignore.txt	stage 4 reviewed exceptions
├─ scripts/	what the pipeline runs – also runnable on a laptop
├─ tools/	
│ └─ install-offline.sh	installs the bundled binaries on an on-prem agent
├─ docs/	this guide (HTML source + PDF)
├─ projects/	
│ └─ <project>/	one folder per team / product area = ownership boundary
│ └─ apis/	
│ └─ <api>.yaml	OpenAPI 3.0.x + x-ibm-configuration, one file for all environments
│ └─ products/	
│ └─ <product>.yaml	plans, rate limits, visibility; apis.*.\$ref → ../apis/<api>.yaml

Why `apis/` and `products/` are separate folders

- A Product must reference its APIs with a relative `$ref`. Putting APIs in a sibling folder makes the reference obvious (`../apis/orders.yaml`) and lets the pipeline map a changed API to the Product that must be republished.
- Both are YAML; the folder tells the pipeline (and the reader) which is which without inspecting content.
- Cross-project references are rejected: a Product can only bundle APIs from its own project. Ownership stays clean.

Conventions enforced automatically

Rule	Enforced by
Files are valid YAML	Stage 1
OpenAPI 3.0.x; <code>info.x-ibm-name</code> is a lowercase slug; <code>info.version</code> is strict semver (MAJOR.MINOR.PATCH); exactly one <code>servers[]</code> entry; <code>x-ibm-configuration</code> present with an explicit <code>enforced</code> ; gateway is <code>datapower-api-gateway</code> ; no callbacks/links	Stage 2
Products reference APIs only via relative <code>\$ref</code> into the same project's <code>apis/</code> ; <code>apic validate</code> passes for the product and every API	Stage 3
No breaking contract change without a MAJOR bump	Stage 4
Environment values live in <code>x-ibm-configuration.properties</code> + <code>catalogs.<catalog>.properties</code> , never in separate files or branches	Code review

5. Life of a change — step by step

#	Who / what	What happens
1	Developer	Creates a branch, edits <code>projects/orders/apis/orders.yaml</code> (say, adds an optional query parameter), opens a pull request to <code>main</code> .
2	Pipeline · <i>validate</i>	Computes the change set against <code>origin/main</code> : the changed API and the Product(s) that <code>\$ref</code> it. Runs stages 1 → 4. Stage 4 compares the file with its version on <code>main</code> .
3	Pull request	The pipeline is a required status check. A red check blocks the merge; annotations point at file and line. The developer fixes and pushes; the pipeline reruns.
4	Reviewer	Reviews the diff like any code. Approves. Merge to <code>main</code> .
5	Pipeline · <i>validate</i> (main)	Same four stages, now against the <code>published/dev</code> tag — i.e. against what dev currently has.
6	Pipeline · <i>publish_dev</i>	Installs/finds tools, sets toolkit credentials, logs in with the API key, recomputes the change set since <code>published/dev</code> , runs the compatibility check against the live catalog, then <code>apic products:publish</code> per Product and verifies the state is <code>published</code> . Moves the <code>published/dev</code> tag to this commit. Logs out.
7	Approver	Azure DevOps Environment <code>apic-test</code> asks for approval. The approver sees exactly which commit and which products are involved.
8	Pipeline · <i>publish_test</i>	Identical to step 6 for catalog <code>test</code> and tag <code>published/test</code> .
9	Approver / Pipeline · <i>publish_prod</i>	Same again for <code>prod</code> . The artifacts are byte-identical to what was tested.

What if nothing under `projects/` changed?

The validate stage reports “no API/product changes detected”, skips stages 1–4 and the publish stages publish nothing. Changing only scripts or configuration therefore never touches API Connect.

What if two merges happen back-to-back?

The second run's change set includes everything since the tag, so it cannot “miss” the first merge. The *Exclusive lock* check on the test and prod Environments serialises publishes so two runs never publish concurrently.

6. The four validation stages

Stage	Tool & command	What it catches	How a failure looks
1 YAML	<code>yamllint -c config/.yamllint.yml</code>	Tabs, duplicate keys, broken indentation, unquoted special characters — anything that stops a parser before it even sees your API.	One annotation per finding with file, line and column.
2 OpenAPI	<code>spectral lint -r config/spectral.yml</code>	Invalid OpenAPI 3.0 structure (unknown fields, bad <code>\$ref</code> , missing responses) plus the APIC house rules: <code>x-ibm-name</code> , <code>semver</code> , <code>single server</code> , <code>enforced</code> , <code>gateway type</code> , <code>no callbacks/links</code> .	Annotations per rule with the rule name (e.g. <code>apic-semver-version</code>) and a JUnit report in the <i>Tests</i> tab.
3 APIC	<code>apic validate <product.yaml></code>	Repository policy first (relative <code>\$ref</code> , file exists, same project). Then the IBM toolkit validates the Product <i>and</i> every referenced API offline: IBM extensions, assembly policies, plan structure.	The toolkit's message, with the product file annotated.
4 Compat	<code>oasdiff breaking <baseline> <new></code>	Contract changes that would break an existing consumer (section 7), compared with the version on <code>main</code> (PR) or the version live in the target catalog (before publish).	A list of breaking changes (rule id, method, path) and one error annotation on the API file.

Example: what a developer sees

```
##[section]Stage 2 - spectral lint (1 file(s))
projects/orders/apis/orders.yaml
  4:12  error  apic-semver-version  info.version must be strict semver (MAJOR.MINOR.PATCH)  info.version
  7:9   error  apic-single-server   The DataPower API Gateway supports a single servers[] entry  servers
##vso[task.logissue type=error;sourcepath=projects/orders/apis/orders.yaml;linenumber=4]apic-semver-version: ...

##[section]Stage 4 - oasdiff breaking, mode=git (1 API(s))
1 changes: 1 error, 0 warning, 0 info
error [api-path-removed-without-deprecation] in API GET /orders/{id}
  api path removed without deprecation
##vso[task.logissue type=error]BREAKING change in orders-api (1.2.0 -> 1.2.1) without a MAJOR version bump.
  Bump MAJOR (new product version) or add a reviewed exception to config/compat-ignore.txt
```

Run the same checks on your laptop

Every stage is a script under `scripts/` with no Azure DevOps dependency. See the README section “Running the checks locally”. The `##vso` lines are harmless noise outside Azure DevOps.

7. Backward compatibility and versioning policy

Stage 4 is the gate that protects the consumers of your APIs. It uses **oasdiff**, which understands OpenAPI 3.0 and ignores the `x-ibm-*` extensions — so changing an assembly policy or a backend URL is never mistaken for a contract change.

What counts as what

Class	Examples	Outcome
Breaking	Remove or rename a path, operation, request parameter or property · add a <i>required</i> request parameter or property · change a type or format · narrow an enum a client sends · remove a response property, status code or media type · tighten security · change the base path	Fails — unless <code>info.version</code> MAJOR was bumped
Potentially breaking	New response status code · new enum value in a response · marking an operation deprecated · loosening response constraints	Reported, does not fail
Non-breaking	New path or operation · <i>optional</i> request parameter or property · new response property · widening an enum a client sends · descriptions, examples, tags · anything under <code>x-ibm-configuration</code>	Passes

How the version number drives the pipeline

Non-breaking change

Keep the version or bump MINOR/PATCH. Same product version. The pipeline **republishes in place** ; existing subscriptions keep working.

VS

Breaking change

Bump the API MAJOR (`1.4.2` → `2.0.0`) and the product version. The pipeline publishes a **new product version that coexists** with the old one. Consumers migrate on their schedule.

Stage 4 compares against the latest published version *with the same MAJOR*. A new MAJOR therefore has no baseline to break and is allowed through (the differences are still printed for the reviewer). Afterwards, subscriptions are moved with `apic products:replace` or `products:supersede` and the old version is retired by hand — that lifecycle is deliberately outside this pipeline.

Exceptions

Sometimes a change that oasdiff calls breaking is acceptable — an endpoint nobody ever subscribed to, or a field documented as internal. Add a line to `config/compat-ignore.txt` **in the same pull request**, as specific as possible (method, path, fragment of the message), with the justification in the PR description. The exception is then reviewed together with the change, and is visible in history forever.

Do not

work around the gate by renaming `x-ibm-name`. That creates a second API and leaves the first one live and unmanaged.

8. Publishing and promotion — dev, test, prod

dev

Environment `apic-dev`

No approval. Publishes on every merge to `main`.

Tag: `published/dev`

test

Environment `apic-test`

Approval + exclusive lock. Same commit as dev.

Tag: `published/test`

prod

Environment `apic-prod`

Approval + exclusive lock. Same commit as test.

Tag: `published/prod`

Inside a publish stage

1. **Tools.** Online agents download them; offline agents must already have them (section 11).
2. **Toolkit credentials.** `credentials.json` from the Secure File → `apic client-creds:set`.
3. **Login.** `apic iam-apikey --server $APIC_SERVER --apiKey $APIC_APIKEY`, followed by a read-only `catalogs:list` as a smoke test. The key comes from a Key Vault-linked variable group and is only ever passed as an environment variable.
4. **Change set.** Everything under `projects/` that changed since this environment's `published/<env>` tag → list of Products.
5. **Live compatibility check.** Stage 4 again, but the baseline is fetched from the catalog with `apic apis:get`. This catches drift caused by anyone who published outside Git.
6. **Publish.** For each Product: `apic products:publish --server ... --org ... --catalog <env> <product.yaml>` (with `--migrate_subscriptions` when the version already exists), then `products:get` must report `state: published`.
7. **Mark.** Move the `published/<env>` tag to this commit and push it.
8. **Always.** Publish logs as a build artifact; `apic logout`.

Why tags instead of “publish what this run validated”?

Because environments fail independently. If prod approval is postponed for a week while dev receives five more merges, the prod stage must publish the union of those changes, not just the last one. Diffing against the environment's own tag gives exactly that, with no bookkeeping outside Git. A missing tag means “never published from Git” and selects every Product — which is also how you bootstrap.

Rolling back

- **Non-breaking regression:** run the pipeline for an earlier commit (*Run pipeline* → *select commit*) and approve the stages you need. The change set is computed relative to the tag, so the older files are republished in place.
- **After a major version:** `apic products:replace` back to the previous product version, then retire the new one.

9. One definition, three environments — catalog-specific properties

The backend URL, timeouts, or a header value usually differ between dev, test and prod. Instead of three copies of the API you declare a **property** with a default and override it per catalog. The gateway substitutes `$(property)` at runtime with the value for the catalog it is running in.

```
x-ibm-configuration:
  enforced: true
  gateway: datapower-api-gateway
  properties:
    target-url:
      value: https://orders-dev.internal.example.com      # default → used by the dev catalog
      description: Backend base URL
  catalogs:                                              # overrides, keyed by catalog name
    test:
      properties:
        target-url: https://orders-test.internal.example.com
    prod:
      properties:
        target-url: https://orders.internal.example.com
  assembly:
    execute:
      - invoke:
          version: 2.0.0
          target-url: $(target-url)$(request.path)      # resolved on the gateway
```

Anything under `x-ibm-configuration` is ignored by the compatibility gate, so switching a backend never counts as a contract change.

Verify once on your tenant

Catalog-specific property overrides are an API Connect feature of the API definition. Confirm on `dev` that the resolved `target-url` is the override, not the default, before relying on it for prod. Alternative if needed: define the property at catalog level in API Manager (Catalog settings → Properties) and reference it the same way.

What must never differ between environments

- Paths, parameters, schemas — the contract. If test and prod need different contracts they are different major versions.
- Security schemes. A prod-only header requirement would be invisible to the compatibility gate on dev.
- Product plans and rate limits — they are part of the reviewed artifact. If prod needs stricter limits, that is a product change reviewed in a PR.

10. Security — identities, secrets, permissions

Item	Where it lives	Notes
IBM Cloud API key (<code>APIC_APIKEY</code>)	Azure Key Vault, surfaced through the variable groups <code>apic-dev</code> , <code>apic-test</code> , <code>apic-prod</code>	One key per environment, each owned by a service ID that is a member of the provider org with a publish role on that catalog only. Masked in logs; only passed via <code>env:</code> .
Toolkit credentials (<code>credentials.json</code>)	Azure DevOps Secure File <code>apic-toolkit-credentials.json</code>	Downloaded from API Manager → <i>Tools for download</i> . Contains the toolkit client id/secret; never committed.
Management endpoint, org name	Variable group <code>apic-shared</code> (plain text)	Not secret, but keep them out of the repository so a fork cannot accidentally target your tenant.
Session token	<code>~/.apiconnect</code> on the agent during the job	<code>apic logout</code> runs in an <code>always()</code> step. On self-hosted agents, run the agent as a dedicated user.
Repository write access (tags)	Build identity	Needs <i>Contribute</i> + <i>Create tag</i> ; on GitHub a token with <code>contents:write</code> . Nothing else is written to Git.
Humans in API Manager	Provider org roles	Viewer on catalogs after go-live. If nobody can click <i>Publish</i> , the live-compatibility check never finds drift.

Never

run `apic --debug` in the pipeline (it can dump tokens), never echo `$APIC_APIKEY`, and never commit `credentials.json`, `.apiconnect/` or `.netrc` — the `.gitignore` blocks the obvious names, not intent.

Rotation

IBM Cloud API keys do not expire unless you set a lifetime. Decide a rotation cadence (e.g. yearly), rotate in Key Vault, and let the next run pick it up — nothing in the repository changes. If your tenant instead requires a toolkit API key created with `apic api-keys:create`, it carries a `ttl`: put the expiry in a calendar, because an expired key fails the login step with an unhelpful message.

11. Setting up an on-prem agent from the offline bundle

The bundle ships everything the pipeline needs on a **Linux x64** agent that has no internet access, except the IBM toolkit (licensed, downloaded from your own tenant).

```
APIConnect-catalog-bundle-<date>/
├─ repo/                               the repository, ready to push to your on-prem Git server
├─ APIConnect-catalog.gitbundle         the same repository with history: git clone APIConnect-catalog.gitbundle
├─ tools/
│   ├── install-offline.sh              installer (also in repo/tools/)
│   └─ linux-x64/
│       ├── bin/  yq · oasdiff · jq · spectral    static binaries; spectral is standalone (no Node.js)
│       ├── wheels/  yamllint · pyyaml · pathspec  for CPython 3.9 – 3.14, pip --no-index
│       ├── SHA256SUMS                      verified by the installer
│       └─ MANIFEST.txt                     versions and sources
└─ docs/  guide.pdf · guide.html
```

Steps

1. **Prepare the machine.** Linux x64 with bash 5, git 2.x, GNU coreutils, Python 3.9+ with pip, and the Azure DevOps agent installed as a dedicated user.
2. **Install the bundled tools.**

```
unzip APIConnect-catalog-bundle-*.zip && cd APIConnect-catalog-bundle-*
sudo tools/install-offline.sh          # verifies SHA256SUMS, installs to /usr/local/bin, pip installs
yamllint
```

3. **Install the IBM toolkit.** In API Manager click *Tools for download*, download the Linux CLI and `credentials.json`.

```
sudo install -m 0755 apic-slim /usr/local/bin/apic    # or `apic` if you downloaded CLI + Designer
apic --accept-license version                          # must match the tenant version, 10.0.8.x
```

4. **Verify as the agent user.** `for t in yamllint spectral yq oasdiff jq apic git realpath; do command -v $t; done` — every line must print a path.
5. **Push the repository** to your on-prem Git (Azure Repos or GitHub Enterprise): `git clone APIConnect-catalog.gitbundle APIConnect-catalog`, add the remote, push `main`.
6. **Tell the pipeline it is offline.** In variable group `apic-shared` set `TOOLS_MODE = offline`. The tooling step then only checks that the tools exist and never reaches the internet; a missing tool fails the job with a clear message.
7. **Point the pipeline at your pool.** Replace `vmImage: ubuntu-latest` with `name: <your agent pool>` in `apic-ci.yml` and `templates/publish-stage.yml`.

Updating tools later

Rebuild the bundle on a connected machine (same layout, regenerate `SHA256SUMS`), copy it over, rerun the installer. Versions are listed in `MANIFEST.txt`; the pipeline prints the versions it found at the start of every job.

12. Setting up Azure DevOps

#	Object	Configuration
1	Pipeline	New pipeline from existing YAML: <code>.azure-pipelines/apic-ci.yml</code> . For Azure Repos add a branch policy on <code>main</code> : <i>Build validation</i> → this pipeline, required. For GitHub the <code>pr</code> : trigger applies; make the check required in branch protection.
2	Variable group <code>apic-shared</code>	<code>APIC_SERVER</code> = management/platform API endpoint of the tenant · <code>APIC_ORG</code> = provider organization name · <code>APIC_TOOLKIT_VERSION</code> = e.g. <code>10.0.8.9</code> · <code>TOOLS_MODE</code> = <code>offline</code> (on-prem) or <code>online</code> .
3	Variable groups <code>apic-dev</code> , <code>apic-test</code> , <code>apic-prod</code>	Linked to Azure Key Vault; each exposes <code>APIC_APIKEY</code> . Grant the pipeline access when prompted on first run.
4	Secure file <code>apic-toolkit-credentials.json</code>	Pipelines → Library → Secure files. Upload the tenant's <code>credentials.json</code> . Authorize the pipeline.
5	Environments <code>apic-dev</code> , <code>apic-test</code> , <code>apic-prod</code>	Pipelines → Environments. On <code>test</code> and <code>prod</code> add <i>Approvals</i> (the people allowed to promote) and <i>Exclusive lock</i> .
6	Agent pool	Self-hosted pool with the prepared Linux agent (section 11), or Microsoft-hosted <code>ubuntu-latest</code> with <code>TOOLS_MODE=online</code> and the toolkit as a Universal Package in an Artifacts feed.
7	Repository permissions	The build identity (<i>Project Collection Build Service</i> or the project's build service) needs <i>Contribute</i> and <i>Create tag</i> on the repository so <code>published/<env></code> tags can be pushed.

First run

1. Open a pull request that touches only the sample project (e.g. edit a description). Confirm the four stages run and the check turns green.
2. Merge. Watch `publish_dev`: it should report “No tag published/dev yet: bootstrapping”, publish every Product in the repository to `dev`, and create the tag.
3. In API Manager, open the `dev` catalog and confirm the product is *Published*. Call the API through the gateway.
4. Approve `test`, then `prod`, when you are ready. Each creates its own tag.
5. Delete `projects/example-project` in a PR once your first real API is in. Remember: deleting the file does not remove the product from API Connect — retire the sample product by hand.

13. Day-2 operations

Add a new API

1. Create `projects/<project>/apis/<api>.yaml` — OpenAPI 3.0.x with `info.x-ibm-name`, semver `info.version`, a single `servers[].url` and an `x-ibm-configuration` block. Copy the sample as a starting point.
2. Either add it to an existing Product (`apis: { <api>: { $ref: ../apis/<api>.yaml } }`) or create a new Product in `products/`.
3. Open a PR. Stage 4 reports “NEW ... no baseline” and passes. Merge, approve.

Make a non-breaking change

Edit the file, optionally bump MINOR/PATCH, open a PR. The Product is republished in place; subscriptions untouched.

Make a breaking change

1. Bump the API `info.version` MAJOR (e.g. `2.0.0`) and the Product `info.version`. Consider a new product file if the old version must stay available for a long time.
2. PR → stage 4 prints the breaking changes but passes because MAJOR changed. Merge, approve.
3. Both versions are now live. Communicate the deprecation; when consumers have moved, migrate remaining subscriptions (`apic products:replace / supersede`) and retire the old version manually.

Change plans or rate limits

Edit the Product file only. It is republished in place; the gateway applies the new limits.

Add a compatibility exception

Add one line to `config/compat-ignore.txt` in the same PR as the change, matching the oasdiff message as specifically as possible (method + path + text fragment). Explain why in the PR.

Retire an API or Product (manual, by design)

1. Remove the files in a PR. The pipeline warns “Deleted in Git but NOT removed from APIC”.
2. In API Manager or with the toolkit: deprecate → retire → delete the product version, per catalog, in the order dev, test, prod.

Roll back

Run pipeline → select an earlier commit, approve the stages you need. For a major version, `products:replace` back and retire the newer version.

Upgrade the tenant / toolkit

IBM upgrades SaaS tenants on their schedule. After an upgrade, download the matching CLI, install it on the agent (offline) or publish it to the Artifacts feed (online) and bump `APIC_TOOLKIT_VERSION`. The version step at the start of each job shows what is in use.

14. Troubleshooting — reading a failed run

Symptom	Meaning and fix
"No API/product changes detected" and everything skipped	Only files outside <code>projects/</code> changed. Expected.
Stage 1 <code>found character '\t' that cannot start any token</code>	A tab in the YAML. Replace with spaces.
Stage 2 <code>apic-x-ibm-name / apic-semver-version</code>	Add <code>info.x-ibm-name</code> (lowercase slug) or make <code>info.version</code> <code>MAJOR.MINOR.PATCH</code> .
Stage 2 <code>apic-single-server</code>	Only one <code>servers[]</code> entry is allowed; its path is the base path.
Stage 3 "must use a relative \$ref ..."	The Product references an API by <code>name:version</code> . Use <code>\$ref: ../apis/<file>.yaml</code> .
Stage 3 <code>apic validate</code> error text	IBM extension or assembly problem; the message is the toolkit's. Run <code>apic validate <product></code> locally to iterate quickly.
Stage 4 "BREAKING change ... without a MAJOR version bump"	Either revert the incompatible part, bump MAJOR, or add a reviewed exception (section 7).
Stage 4 "oasdiff failed (exit 2)"	<code>oasdiff</code> could not load one of the specs — usually a <code>\$ref</code> to a missing component. Fix the spec; stage 2 normally catches this first.
Publish: "offline mode: 'X' is not installed on this agent"	Run <code>tools/install-offline.sh</code> on the agent, or install the toolkit; check <code>PATH</code> for the agent user.
Publish: login fails	Check <code>APIC_SERVER</code> , that the API key's service ID is a member of the provider org, and that the key is not expired. Try the same command interactively from the agent.
Publish: live compatibility check fails but the PR passed	What is live in the catalog differs from what Git thinks — someone published outside Git. Reconcile: export the live definition, compare, decide, then republish from Git.
Publish: "ended in state 'staged'/'pending'"	The product uploaded but did not activate. Check the catalog's gateway service and any approval/task settings in Catalog settings.
Publish: <code>git push ... refs/tags/published/...</code> permission denied	Grant the build identity <i>Contribute</i> + <i>Create tag</i> . The publish itself already happened; rerun the stage to set the tag.
Publish to <code>prod</code> rejected for an existing version	The prod catalog may be in <i>Production mode</i> , which can block republishing the same version. Either disable it or adopt "every prod change bumps the version + <code>products:replace</code> ".

15. Before the first publish — items to verify on your tenant

The pipeline uses commands documented for the v10.0.8 toolkit. A few behaviours depend on your tenant and were flagged in the design; check them once from a laptop with the toolkit installed.

	Check	How
☐	Non-interactive login works	<code>apic iam-apikey --server <mgmt> --apiKey <IBM Cloud API key></code> then <code>apic catalogs:list --server ... --org ...</code> . If the tenant only accepts IBMid/OIDC, switch the login step to <code>apic login --server ... --sso --context provider --apiKey <toolkit key from apic api-keys:create></code> .
☐	Shape of <code>apis:get</code> output	<code>apic apis:get --scope catalog <name>:<version> --catalog dev --org ... --server ... --format yaml --output -</code> . The script accepts the bare definition or an envelope with <code>.api</code> ; look at the first live-compat log.
☐	Shape of <code>apis:list --format json</code>	Expected <code>{ "results": [...] }</code> ; the script also accepts a bare array.
☐	<code>apic validate</code> exit code	Break a sample on purpose; the command must exit non-zero. The script additionally counts “Validated” lines.
☐	<code>--migrate_subscriptions</code> on republish	Republish an existing version in <code>dev</code> with a test subscription and confirm it survives.
☐	Catalog-specific properties resolve	Publish the sample to <code>dev</code> and <code>test</code> ; the invoke target must differ.
☐	Production mode on <code>prod</code>	Catalog settings. Decide the policy (section 14, last row).
☐	Gateway OpenAPI 3.0 subset	Optionally run <code>apic draft-apis:validate</code> online in <code>dev</code> for one representative API.
☐	Existing content in the catalogs	Anything published today that is not in Git will be flagged as drift by the live check, or silently replaced if a product with the same <code>name:version</code> is in Git. Export or retire it first (your manual “start clean”).

16. Appendix — scripts, tools and versions

Scripts

Script	Role
<code>scripts/lib.sh</code>	Shared helpers: logging, Azure DevOps annotations, list reading, <code>\$ref</code> resolution.
<code>scripts/changed-products.sh</code> <code>BASE HEAD OUT</code>	Stage 0. Git diff → Product list; deleted files → warning only.
<code>scripts/product-apis.sh --</code> <code>list FILE</code>	Product list → referenced API files.
<code>scripts/validate-yaml.sh</code> <code>LIST...</code>	Stage 1, yamllint.
<code>scripts/validate-oas.sh APIS</code>	Stage 2, spectral; writes <code>out/spectral.xml</code> .
<code>scripts/validate-apic.sh</code> <code>PRODUCTS</code>	Stage 3, <code>\$ref</code> policy + <code>apic validate</code> .
<code>scripts/compat-check.sh --</code> <code>mode git live ...</code>	Stage 4 / 4', oasdiff; writes <code>out/compat/*.xml</code> .
<code>scripts/publish.sh --catalog</code> <code>C PRODUCTS</code>	<code>apic products:publish</code> + state assertion.
<code>tools/install-offline.sh</code>	Installs the bundled binaries and yamllint on an on-prem agent.

Bundled tool versions (Linux x64)

Tool	Version	Purpose
yq	v4.53.6	YAML queries in scripts (mikefarah/yq)
oasdiff	v1.32.1	Stage 4 backward-compatibility
jq	1.8.2	JSON parsing of toolkit output
spectral	v6.16.3 (standalone)	Stage 2 OpenAPI lint; no Node.js needed
yamllint	1.38.0 (1.37.1 for Python 3.9)	Stage 1 YAML syntax; wheels for CPython 3.9–3.14
apic toolkit	10.0.8.x — not bundled	Download from your tenant (API Manager → Tools for download)

Toolkit commands used (v10.0.8 CLI reference)

`apic client-creds:set` · `apic iam-apikey` (or `apic login --sso --apiKey`) · `apic catalogs:list` · `apic validate` ·
`apic apis:list` · `apic apis:get` · `apic products:publish` · `apic products:get` · `apic logout` · Manual lifecycle: `apic`
`products:replace` · `products:supersede` · `products:delete`.